

ADVANCED SQL QUERIES USING JOINS AND SUBQUERIES

ASSIGNMENT 9

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: TINAKAR C

REGISTER NO: 2511021240003

COURSE: BCA

YEAR: 2ND YEAR

INTRODUCTION

Advanced SQL queries retrieve meaningful information from multiple related tables. Joins combine records through related columns, while subqueries use the result of one query within another. This assignment uses MySQL joins and subqueries to prepare reports about online shopping customers, products, orders, and payments.

1. REPORT ON CUSTOMER ORDERS USING INNER JOIN

INNER JOIN combines rows when the related column values match.

```
SELECT O.order_id, C.customer_name, O.order_date,
       O.total_amount, O.order_status
FROM ORDERS O
INNER JOIN CUSTOMER C ON O.customer_id = C.customer_id
ORDER BY O.order_id;
```

Output:

order_id	customer_name	order_date	total_amount	order_status
401	Aarav Kumar	2026-10-15	2298.00	Delivered
402	Diya Sharma	2026-10-16	1497.00	Delivered
403	Rohan Verma	2026-10-17	1300.00	Delivered
404	Meera Reddy	2026-10-18	2499.00	Processing
405	Kiran Raj	2026-10-19	1199.00	Confirmed

This query joins ORDERS and CUSTOMER to display each order with the customer's name and order details.

2. REPORT ON ORDERED PRODUCTS USING MULTIPLE JOINS

This report displays the products included in each order, along with customer name, quantity, and unit price.

```
SELECT O.order_id, C.customer_name, P.product_name,
       OI.quantity, OI.unit_price,
       OI.quantity * OI.unit_price AS line_total
FROM ORDERS O
INNER JOIN CUSTOMER C ON O.customer_id = C.customer_id
INNER JOIN ORDER_ITEM OI ON O.order_id = OI.order_id
INNER JOIN PRODUCT P ON OI.product_id = P.product_id
ORDER BY O.order_id, OI.order_item_id;
```

Output:

order_id	customer_name	product_name	quantity	unit_price	line_total
401	Aarav Kumar	Wireless Mouse	1	799.00	799.00
401	Aarav Kumar	Bluetooth Headphones	1	1499.00	1499.00
402	Diya Sharma	Cotton T-Shirt	3	499.00	1497.00
403	Rohan Verma	DBMS Fundamentals	2	650.00	1300.00
404	Meera Reddy	Kitchen Mixer	1	2499.00	2499.00
405	Kiran Raj	Laptop Backpack	1	1199.00	1199.00

The query joins four tables to show which products each customer ordered and calculates the line total.

3. REPORT ON CATEGORIES AND THEIR PRODUCT COUNTS USING LEFT JOIN

LEFT JOIN returns all rows from the left table, including categories without matching products.

```
SELECT C.category_id, C.category_name,
       COUNT(P.product_id) AS total_products
FROM CATEGORY C
LEFT JOIN PRODUCT P ON C.category_id = P.category_id
GROUP BY C.category_id, C.category_name
ORDER BY C.category_id;
```

Output:

category_id	category_name	total_products
10	Electronics	3
20	Fashion	2
30	Home & Kitchen	2
40	Books	1

This query lists each category and counts its products. LEFT JOIN would also keep categories with zero products.

4. PRODUCTS PRICED ABOVE THE AVERAGE USING A SUBQUERY

A subquery is a query nested inside another SQL statement.

```
SELECT product_name, price
FROM PRODUCT
WHERE price > (SELECT AVG(price) FROM PRODUCT)
ORDER BY price DESC;
```

Output:

product_name	price
Kitchen Mixer	2499.00
Bluetooth Headphones	1499.00
Laptop Backpack	1199.00

The inner query calculates the average product price. The outer query returns products priced above that average.

5. ORDERS WITH SUCCESSFUL PAYMENTS USING INNER JOIN

```
SELECT O.order_id, C.customer_name, P.payment_method,
       P.amount, P.payment_status
FROM ORDERS O
INNER JOIN CUSTOMER C ON O.customer_id = C.customer_id
INNER JOIN PAYMENT P ON O.order_id = P.order_id
WHERE P.payment_status = 'Successful'
ORDER BY O.order_id;
```

Output:

order_id	customer_name	payment_method	amount	payment_status
401	Aarav Kumar	UPI	2298.00	Successful
402	Diya Sharma	Card	1497.00	Successful
403	Rohan Verma	UPI	1300.00	Successful

This query combines orders, customers, and payment records, then filters for successful payments.

6. CUSTOMERS WHO HAVE PLACED ORDERS USING EXISTS

```
SELECT C.customer_id, C.customer_name
FROM CUSTOMER C
WHERE EXISTS (
  SELECT 1 FROM ORDERS O
  WHERE O.customer_id = C.customer_id
)
ORDER BY C.customer_id;
```

Output:

customer_id	customer_name
1	Aarav Kumar
2	Diya Sharma
3	Rohan Verma
4	Meera Reddy
5	Kiran Raj

EXISTS checks whether a matching order exists for each customer.

CONCLUSION

INNER JOIN and LEFT JOIN combine related shopping records, while subqueries and EXISTS support more advanced filtering. These reports help monitor customer orders, product sales, category inventories, and payment status in the Online Shopping Management System.